

Optimizing Multi-Robot Delivery in Smart Hotels: A Multi-Objective Metaheuristic Approach

Seif Abdallah¹, Yahia Abdelraheem¹, Nour Eldin Daoud¹, Abdelrahman Mohamed Wael¹,
Ahmed Elkot¹, Youssef Maged¹, Jessica Magdy^{2,3} and Omar Shehata^{2,3}

¹Computer Science Department, Faculty of Media Engineering and Technology, German University in Cairo, Egypt

²Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt

³Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

{abdelrahman.farouk, seif.sayed, yahia.abdelraheem, noureldin.dawod, ahmed.elkot, youssef.nazir}@student.guc.edu.eg,
{jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—This paper addresses the multi-robot task allocation problem in multi-floor indoor service environments by evaluating and comparing several metaheuristic algorithms, including Simulated Annealing (SA), Genetic Algorithm (GA), Ant Colony Optimization (ACO), and Multi-Objective Artificial Bee Colony (MOABC). The problem is formulated as a multi-objective optimization that simultaneously minimizes total travel distance, completion time, and workload imbalance among heterogeneous robots operating across multiple floors. Through multiple independent runs, the results demonstrate that MOABC consistently achieves superior solution quality with the lowest variability, while GA provides a favorable trade-off between optimality and computational efficiency. SA delivers near-instantaneous convergence, whereas ACO exhibits limited scalability. All evaluated approaches yield a substantial reduction in workload variance, confirming the effectiveness of population-based metaheuristics for multi-objective task allocation in service robotics.

Index Terms—Multi-robot task allocation, metaheuristic optimization, multi-objective optimization, service robotics, workload balancing.

I. INTRODUCTION

Autonomous delivery robots are transforming service operations in smart environments, particularly in the hospitality sector [1]. In large hotels, multi-robot delivery systems offer a reliable alternative to human-based delivery, autonomously navigating corridors, elevators, and multiple floors to deliver room service orders, linens, and amenities.

Efficient task allocation among multiple robots is critical in complex hotel environments. Poor task distribution leads to delays, congestion, and inefficient energy usage. Intelligent task allocation is required to assign delivery tasks optimally, minimizing delivery time while balancing robot workloads. Metaheuristic optimization techniques have been widely employed in AGV and robotic systems for path planning and task allocation [2], [3]. To address these challenges, we explore Simulated Annealing (SA) [4], Genetic Algorithms (GA) [5], Ant Colony Optimization (ACO) [6], and Artificial Bee Colony (ABC) [7] for minimizing total task completion time, energy consumption, and load imbalance. The primary aim is to minimize total travel time while ensuring efficient

coordination subject to physical, kinematic, and collision-avoidance constraints.

II. LITERATURE REVIEW

Game-theoretic and hybrid control frameworks [8] enhance cooperative decision-making via potential fields, GA, and fuzzy logic, but incur high computational costs. Adaptive evolutionary algorithms such as AMOEAD-HRM [9] optimize completion time, energy, and load balance in dynamic warehouses through predictive memory strategies. Graph-based and vehicle-routing formulations [10], [11] address multi-floor navigation with elevator scheduling and Adaptive Large Neighborhood Search under time-window constraints. Marginal-cost heuristics with Large Neighborhood Search [12] demonstrate practical feasibility in large-scale hotel deployments but assume deterministic operating conditions. Scalability to larger robot fleets and unified multi-objective frameworks without extensive parameter tuning remain open challenges that this paper addresses.

III. METHODOLOGY

A. Problem Formulation

The multi-robot hotel delivery problem is formulated to optimize task allocation and execution sequencing across K robots operating on multiple floors connected by elevators. Each robot k performs a sequence of m_k tasks distributed at different floor levels, with the primary objectives of minimizing the total travel distance, balancing the distribution of workload, and reducing the maximum completion time.

A task sequence for robot k is defined as an ordered list of tasks starting from the depot and ending at the final task location:

$$s_k = [s_{k,0}, s_{k,1}, s_{k,2}, \dots, s_{k,m_k}] \quad (1)$$

where $s_{k,0}$ denotes the depot location for robot k , and m_k is the number of tasks assigned to robot k . The complete solution is represented as a matrix containing all robot task sequences, where each robot's route is encoded as an ordered list of task identifiers with their corresponding floor locations.

The decision variables for this optimization problem are the binary task assignment variable $y_{tk} \in \{0, 1\}$, which equals 1 if task t is assigned to robot k and 0 otherwise, and the integer task ordering variable $o_{kt} \in \mathbb{Z}^+$, representing the execution order of task t in robot k 's route. The distance between two consecutive tasks i and j on the same floor is calculated using the Euclidean distance:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (2)$$

where (x_i, y_i) and (x_j, y_j) are the planar coordinates of tasks i and j , respectively, in metres. All times are in seconds; robot speed is $v = 2.5 \text{ m s}^{-1}$. When tasks are on different floors, an additional elevator transition time $t_{f_i \rightarrow f_j}$ is incurred. This is modelled as a fixed cost of 3 s per inter-floor trip (representing waiting plus vertical travel for light-duty hotel lifts), as opposed to a stochastic model, since no empirical waiting-time distribution is assumed.

The total planar travel distance for robot k is computed as:

$$D_k = \sum_{i=1}^{m_k-1} \left[(1-\delta_i) \cdot d_{s_{k,i}, s_{k,i+1}} + \delta_i \cdot (d_{s_{k,i}}^{\text{to-elev}} + d_{s_{k,i+1}}^{\text{from-elev}}) \right] \quad (3)$$

where $d_{s_{k,i}}^{\text{to-elev}}$ is the planar distance from task $s_{k,i}$ to the elevator on its floor, $d_{s_{k,i+1}}^{\text{from-elev}}$ is the planar distance from the elevator to task $s_{k,i+1}$ on the destination floor, and δ_i is a floor-change indicator:

$$\delta_i = \begin{cases} 1, & \text{if } f_{s_{k,i}} \neq f_{s_{k,i+1}}, \\ 0, & \text{otherwise,} \end{cases}$$

When tasks are on the same floor ($\delta_i = 0$), the direct Euclidean distance $d_{s_{k,i}, s_{k,i+1}}$ is used. When tasks are on different floors ($\delta_i = 1$), the distance accounts for walking to and from the elevator instead.

The total elevator-related transition time for robot k accounts for waiting and vertical travel only, since walking to and from the elevator is already captured in D_k :

$$T_k^{\text{elev}} = \sum_{i=1}^{m_k-1} \delta_i \cdot t_{f_{s_{k,i}} \rightarrow f_{s_{k,i+1}}} \quad (4)$$

where $t_{f_{s_{k,i}} \rightarrow f_{s_{k,i+1}}}$ is the elevator waiting and vertical travel time between floors. The execution time for robot k includes planar navigation time, elevator transition time, and service time:

$$T_k = \frac{D_k}{v} + T_k^{\text{elev}} + \sum_{t \in T} \tau_t y_{tk} \quad (5)$$

where v is the constant robot velocity (m/s), T is the task set, and τ_t is the service time of task t (s).

The optimization problem simultaneously addresses three competing objectives. The first objective minimizes the total travel distance across all robots:

$$J_1 = \sum_{k=1}^K D_k \quad (6)$$

The second objective minimizes the maximum completion time (makespan) to balance robot operation durations:

$$J_2 = \max_{k \in R} T_k \quad (7)$$

where R denotes the robot set.

The following constraint limits the operational time of each robot:

$$\frac{D_k}{v} + T_k^{\text{elev}} + \sum_{t \in T} \tau_t y_{tk} \leq Q, \quad \forall k \in R \quad (8)$$

where $Q=500$ is the maximum allowable composite workload (battery endurance or shift limit). Infeasible solutions are penalised by 10^6 rather than discarded, so the constraint is soft.

The third objective minimizes the workload imbalance measured as the normalized standard deviation of robot workloads. The workload for robot k is defined as:

$$E_k = \gamma_1 \sum_{t \in T} \tau_t y_{tk} + \gamma_2 D_k \quad (9)$$

where γ_1 and γ_2 weight the service time and travel distance contributions, respectively.

The average workload across all robots is:

$$E_{\text{avg}} = \frac{1}{K} \sum_{k=1}^K E_k \quad (10)$$

The workload imbalance objective is then:

$$J_3 = \frac{\sqrt{\frac{1}{K} \sum_{k=1}^K (E_k - E_{\text{avg}})^2}}{E_{\text{avg}}} \quad (11)$$

The combined multi-objective function aggregates these three objectives using weight coefficients:

$$\min J = \alpha * J_1 + \beta_1 * J_2 + \beta_2 * J_3 \quad (12)$$

where α , β_1 , and β_2 are tunable weights. In all experiments, $(\alpha, \beta_1, \beta_2) = (0.75, 0.80, 1600)$; the large β_2 compensates for the dimensionless scale of J_3 relative to J_1 and J_2 .

The optimization is subject to several constraints ensuring feasibility and operational validity. Each task must be assigned to exactly one robot:

$$\sum_{k=1}^K y_{tk} = 1, \quad \forall t \in T \quad (13)$$

Each robot must respect maximum workload capacity to prevent battery depletion or operational overload:

$$\sum_{t \in T} \tau_t * y_{tk} + \frac{1}{v} * \sum_{i=1}^{|o_k|-1} d_{o_i^k, o_{i+1}^k} \leq Q, \quad \forall k \in R \quad (14)$$

where Q represents the maximum allowable workload per robot. Additionally, all decision variables must satisfy domain constraints with $y_{tk} \in \{0, 1\}$ and $o_{kt} \in \mathbb{Z}^+$.

This formulation captures the core task-to-robot assignment and sequencing decisions, abstracting path planning into inter-task distances and elevator penalties. The multi-objective structure enables flexible prioritization of distance, completion time, and workload fairness.

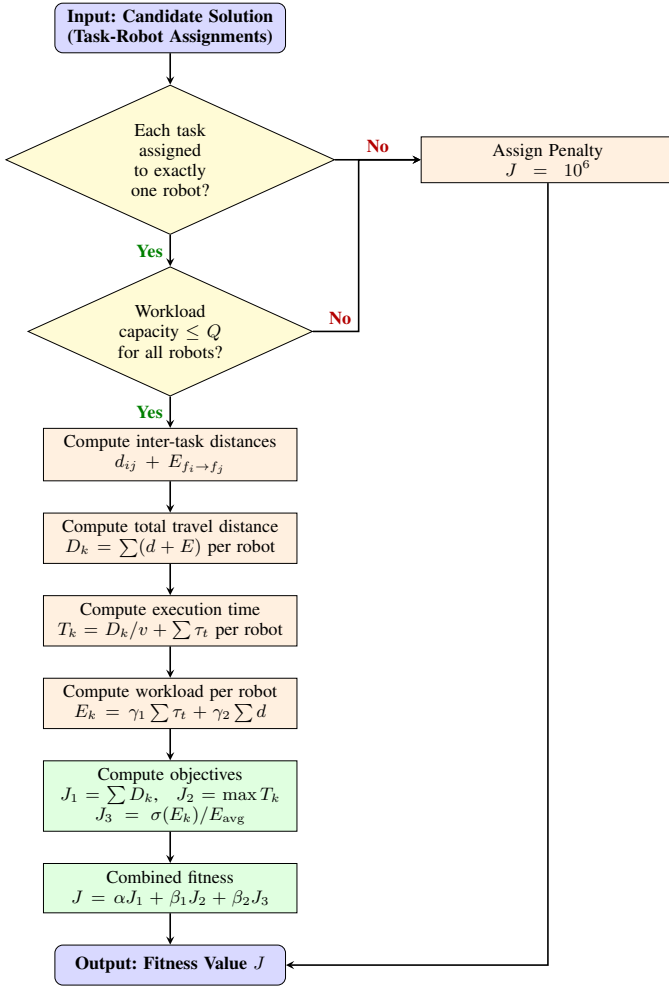


Fig. 1: Solution evaluation process for the multi-robot task allocation framework.

Figure 1 presents the complete solution evaluation workflow, beginning with constraint verification and progressing through workload computation to final objective calculation.

B. Simulated Annealing

SA [4] uses a probabilistic acceptance mechanism to escape local optima in the combinatorial task-allocation space. Starting from an initial feasible solution s_0 , the algorithm explores neighboring solutions through four perturbation operators: intra-robot swap, inter-robot swap, task reassignment, and insertion move. The cost of each candidate is evaluated using the combined objective function in Eq. (12). A linear cooling schedule $T_k = T_0 - k \cdot \Delta T$ progressively reduces the acceptance probability of worse solutions. New solutions are accepted with probability $P = 1$ if $\Delta J < 0$, or $P = \exp(-\Delta J/T)$ otherwise, where $\Delta J = J(s') - J(s)$. The algorithm terminates when the temperature falls below a minimum threshold or a maximum iteration count is reached.

C. Genetic Algorithm

The Genetic Algorithm [5] evolves a population of candidate solutions through selection, crossover, and mutation.

Each individual encodes ordered task sequences per robot; invalid individuals receive a penalty of 10^6 . Three selection operators are used: tournament, roulette-wheel, and best-rank. A hybrid Order-Crossover / Two-Point-Crossover operator with a repair step preserves task-assignment validity. Four mutation operators—intra-robot swap, inter-robot swap, task reassignment, and insertion move—diversify the population. Elitism retains top individuals across generations, and the algorithm terminates after a predefined number of generations [13].

D. Ant Colony Optimization

Ant Colony Optimization [6] is a swarm intelligence meta-heuristic where pheromone trails guide probabilistic construction of task-to-robot assignments. Each ant constructs a complete solution by iteratively assigning tasks to robots based on probabilistic selection. The probability that task t is assigned to robot k is calculated as:

$$P_{kt} = \frac{[\tau_{kt}]^\alpha * [\eta_{kt}]^\beta}{\sum_{r \in R} [\tau_{rt}]^\alpha * [\eta_{rt}]^\beta} \quad (15)$$

where τ_{kt} is the pheromone level representing learned desirability from previous successful solutions, $\eta_{kt} = 1/d_{kt}$ is the heuristic desirability based on inverse distance (tasks closer to a robot's current position are preferred), α controls pheromone influence, and β controls heuristic influence. This probability is compared to a random number $r \sim \text{Uniform}(0, 1)$ using roulette-wheel selection.

After all ants construct solutions, pheromones are updated through evaporation and reinforcement:

$$\tau_{kt} \leftarrow (1 - \rho) * \tau_{kt} + \Delta \tau_{kt} \quad (16)$$

where the evaporation rate ρ prevents unlimited accumulation and allows the algorithm to forget poor patterns, while the pheromone deposit $\Delta \tau_{kt} = 1/(J_4^{\text{best}} + \epsilon)$ inversely relates to solution cost, reinforcing better task-robot assignments. The algorithm terminates after a predefined number of iterations.

E. Artificial Bee Colony

The Multi-Objective ABC (MOABC) variant [7] maintains a Pareto archive of non-dominated solutions across all three objectives (J_1, J_2, J_3), enabling trade-off analysis without fixed scalarization weights. The colony (size 20) is divided into three bee types operating each iteration. *Employed bees* each own one food source; they perturb it with one of the four mutation operators and update the archive if the neighbor is non-dominated. *Onlooker bees* select food sources with probability proportional to crowding distance in the current archive, then apply the same perturbation, biasing search toward sparse Pareto regions. A per-source trial counter increments whenever a perturbation fails to improve the archive; once it exceeds the abandonment limit ($L = 10$), a *scout bee* discards that source and initializes a new random feasible solution. The archive is pruned by crowding-distance ranking whenever it exceeds the colony capacity.

F. Implementation Details

All algorithms are implemented in Python 3.14 (NumPy 2.4, SciPy 1.17) and executed single-threaded on one core of a 12th-Gen Intel Core i5-1235U (1.3 GHz base / 4.4 GHz boost, 10 cores, 32 GB RAM, Windows 11). Workload weights $\gamma_1 = \gamma_2 = 1.0$ assign equal importance to service time and travel distance in E_k ; the scalarization weights $(\alpha, \beta_1, \beta_2)$ were chosen by a pilot grid search on the small instance. Table I summarises all hyperparameters. Each algorithm uses seed $42 + k$ for trial k ($k = 0, \dots, 9$), yielding fully reproducible results.

TABLE I: Hyperparameters Used in All Experiments

Parameter	Value	Scope
α, β_1, β_2	0.75, 0.80, 1600	SA, GA, ACO
$\gamma_1, \gamma_2; Q$	1.0, 1.0; 500	All
SA: T_0 , iters	200, 1000	SA
GA: pop. gen. p_c/p_m	10, 200, 0.7/0.1	GA
ACO: ants, iters, $\rho, \alpha_a/\beta_a$	10, 50, 0.3, 1/2	ACO
MOABC: colony, iters, L	20, 100, 10	MOABC
NSGA-II: pop. gen. p_c/p_m	20, 200, 0.9/0.3	NSGA-II

IV. RESULTS AND FINDINGS

A. Single-Run Algorithm Results

Figure 2 shows SA reducing J_4 on the medium instance with J_3 falling sharply as workloads balance, demonstrating effective escape from the highly imbalanced initial solution. GA (Figure 3) converges to a low-imbalance allocation within 200 generations. ACO (Figure 4) converges earlier due to pheromone saturation by iteration 50, limiting further exploration. MOABC (Figure 5) generates a Pareto archive covering distance-optimized, balance-optimized, and makespan-optimized trade-offs.

Simulated Annealing Optimization Progress

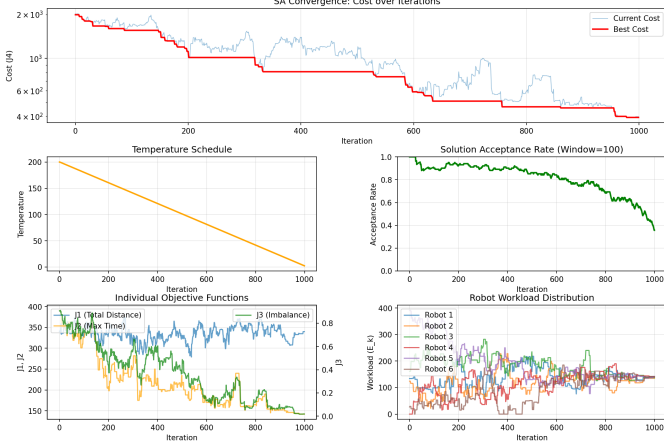


Fig. 2: SA optimization progress over 1000 iterations.

B. Comparative Analysis

All five algorithms were executed for 10 independent trials per instance (seeds 42–51) on an Intel Core i5-1235U / Windows 11 (see Section III-F). NSGA-II employs fast non-dominated sorting with crowding-distance selection; its operators are shared with GA for a fair comparison, and its

Genetic Algorithm Optimization Progress

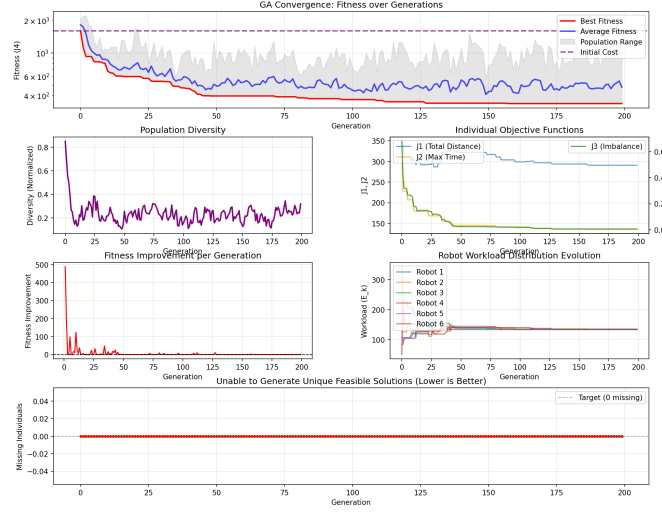


Fig. 3: GA optimization progress over 200 generations.

Ant Colony Optimization — Convergence Report

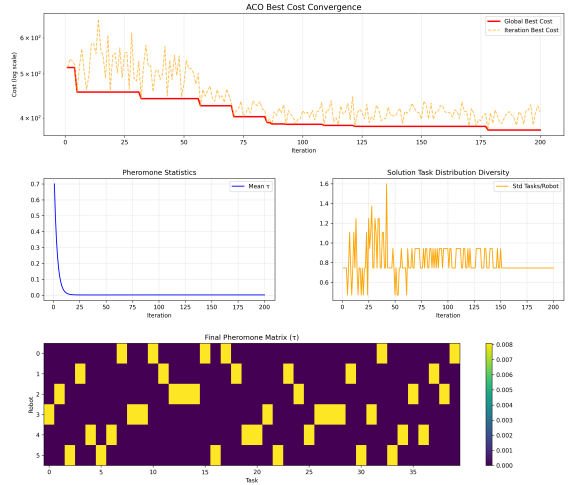


Fig. 4: ACO convergence over 50 iterations.

Pareto front averaged 20 non-dominated solutions per trial. Table II reports mean $\pm$ std of J_4 and average CPU time; boldface marks the best per instance. ACO runtime grows as $\mathcal{O}(\text{ants} \times \text{tasks} \times \text{iters})$ due to per-ant full-task enumeration, explaining its 48.7 s at XLarge vs. ≤ 0.17 s for SA.

NSGA-II achieves the best scalar J_4 on four of five instances (Small through XLarge), improving over the second-best algorithm by 12.3–28.5%. On the tiny instance, MOABC achieves the lowest mean (57.14) owing to its dense Pareto archive on small search spaces. SA consistently delivers the fastest computation (≤ 0.30 s per trial) but the highest variability. ACO scales poorly in time due to per-ant tour construction; MOABC offers competitive quality with moderate runtime

TABLE II: Scalar Combined Fitness J_4 (Mean $\pm$ Std) and Average CPU Time per Trial across Five Benchmark Instances (10 Trials Each)

Instance	SA	GA	ACO	MOABC	NSGA-II
Tiny (2R, 9T, 3F)	65.84 $\pm$ 4.26	70.17 $\pm$ 4.02	71.07 $\pm$ 2.80	57.14 $\pm$ 1.68	59.75 $\pm$ 0.52
Small (3R, 15T, 3F)	119.94 $\pm$ 7.99	115.20 $\pm$ 6.44	118.76 $\pm$ 10.23	102.13 $\pm$ 6.92	87.64 $\pm$ 9.90
Medium (6R, 40T, 5F)	362.16 $\pm$ 22.91	325.85 $\pm$ 11.43	402.25 $\pm$ 16.68	348.17 $\pm$ 23.19	253.25 $\pm$ 15.45
Large (10R, 80T, 8F)	910.57 $\pm$ 97.39	707.31 $\pm$ 26.35	854.83 $\pm$ 23.02	762.33 $\pm$ 18.25	620.18 $\pm$ 34.76
XLarge (15R, 120T, 10F)	1709.79 $\pm$ 136.32	1166.28 $\pm$ 42.36	1386.68 $\pm$ 18.74	1314.94 $\pm$ 48.22	1088.21 $\pm$ 35.31
Average CPU time per trial (seconds)					
Tiny	0.02	0.14	0.39	0.53	1.92
Small	0.10	0.56	2.79	1.05	1.87
Medium	0.18	0.99	15.44	1.76	2.39
Large	0.30	1.67	25.03	1.09	1.39
XLarge	0.17	1.17	48.69	1.44	1.76

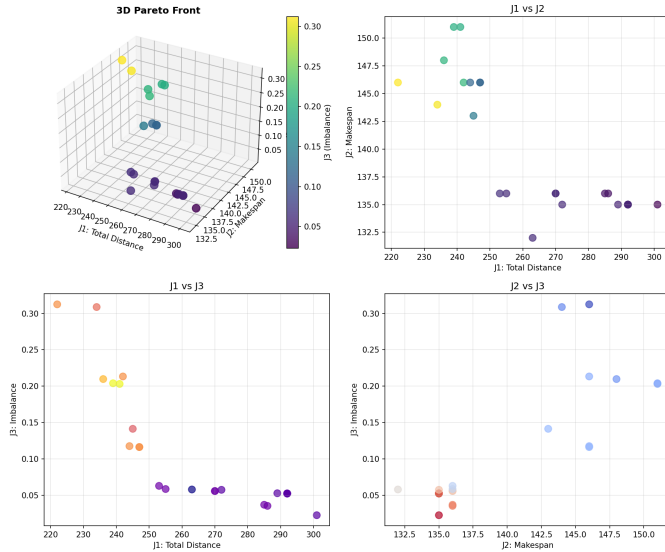


Fig. 5: MOABC 3-D Pareto front (medium instance): non-dominated solutions spanning distance-optimized (J_1), balance-optimized (J_3), and makespan-optimized (J_2) trade-offs.

across all scales.

C. Multi-Objective Quality Metrics

Table III compares MOABC and NSGA-II using Hypervolume (HV) and Inverted Generational Distance (IGD), averaged over 10 trials. A shared reference point at $1.1 \times$ the nadir vector is used for HV; IGD is computed against the combined non-dominated reference front pooled across both algorithms and all trials. Higher HV and lower IGD indicate better Pareto-front quality.

TABLE III: Multi-Objective Quality Metrics: HV and IGD (Mean $\pm$ Std, 10 Trials)

Instance	MOABC		NSGA-II	
	HV	IGD	HV	IGD
Tiny	2.272 $\pm$ 0.113	0.041 $\pm$ 0.050	2.008 $\pm$ 0.044	0.158 $\pm$ 0.051
Small	1.544 $\pm$ 0.093	0.242 $\pm$ 0.044	1.908 $\pm$ 0.095	0.074 $\pm$ 0.050
Medium	0.996 $\pm$ 0.074	0.472 $\pm$ 0.045	1.791 $\pm$ 0.141	0.113 $\pm$ 0.045
Large	1.059 $\pm$ 0.046	0.450 $\pm$ 0.030	1.868 $\pm$ 0.139	0.154 $\pm$ 0.051
XLarge	0.933 $\pm$ 0.123	0.492 $\pm$ 0.060	1.841 $\pm$ 0.174	0.090 $\pm$ 0.045

On the tiny instance, MOABC's archive (mean 19 solutions) produces a denser front with higher HV and lower IGD. From small scale onward, NSGA-II's elitist crowding-distance selection maintains better spread, yielding consistently higher HV and lower IGD. MOABC's IGD deteriorates at medium-to-large scales (0.45–0.49), reflecting insufficient archive coverage relative to the expanding objective space.

D. Statistical Significance

The Friedman non-parametric test assesses whether algorithm rankings differ significantly. A pairwise Wilcoxon signed-rank test with Bonferroni correction (adjusted $\alpha = 0.005$ for 10 pairs) is applied post-hoc. Table IV summarises results; all five instances yield $p < 10^{-6}$.

TABLE IV: Friedman Test Results (10 Trials, 5 Algorithms)

Instance	Friedman χ^2	p -value
Tiny	36.18	2.65×10^{-7}
Small	30.96	3.12×10^{-6}
Medium	34.88	4.92×10^{-7}
Large	36.08	2.79×10^{-7}
XLarge	38.72	7.96×10^{-8}

Post-hoc Wilcoxon tests (Bonferroni-corrected, $\alpha=0.005$) confirm the following. On the *tiny* instance, MOABC is significantly superior to SA, GA, and ACO; SA is also significantly superior to both GA and ACO. On the *small* instance, NSGA-II and MOABC are each significantly superior to SA, GA, and ACO; MOABC vs. NSGA-II is not significant. At *medium* scale, NSGA-II is significantly superior to all four others; ACO is significantly inferior to GA, MOABC, and NSGA-II. At *large* scale, NSGA-II significantly outperforms all others; GA significantly outperforms ACO and MOABC. At *XLarge* scale (9 of 10 pairs significant), SA is the clear outlier; GA vs. NSGA-II is the only non-significant pair, reflecting convergence of population-based methods at high task counts.

V. CONCLUSION

This paper evaluated five optimization algorithms for multi-robot task allocation in multi-floor hotel environments, minimising total travel distance (J_1), makespan (J_2), and workload imbalance (J_3), with J_2 correctly computed as the maximum actual robot completion time. Experiments spanned five benchmark instances (9–120 tasks, 2–15 robots, 3–10 floors) with 10

independent trials each, assessed via Friedman and Bonferroni-corrected Wilcoxon tests.

NSGA-II achieved the best scalarised fitness J_4 on four of five instances, outperforming the next-best by 12.3–28.5% at medium-to-XLarge scale while remaining under 2.4 s per trial. On the tiny instance (9 tasks), MOABC's dense Pareto archive yielded the lowest mean $J_4 = 57.14$. From a multi-objective perspective, NSGA-II produced superior Hypervolume (1.79–1.91) and IGD (0.07–0.16) at 15+ tasks; MOABC's IGD degraded at larger scales (up to 0.49). SA delivered the fastest computation (≤ 0.30 s) but the highest variance, while ACO scaled poorly in time (up to 48.7 s per trial at XLarge).

Friedman tests confirmed statistically significant ranking differences on all five instances ($p < 10^{-6}$). These results establish NSGA-II as the recommended algorithm for quality-critical multi-robot hotel task allocation at medium-to-large scale, with MOABC preferred for small instances requiring diverse Pareto-front coverage.

VI. FUTURE WORK

Several directions can extend the current framework. First, *dynamic task allocation* would allow new service requests to arrive in real time while robots are in motion, requiring online replanning mechanisms such as event-driven NSGA-II restarts or incremental Pareto-archive updates. Second, the current formulation uses a fixed weighted scalarisation ($J_4 = \alpha J_1 + \beta_1 J_2 + \beta_2 J_3$) to embed the multi-objective problem in a single-objective space for SA, GA, and ACO; replacing this with adaptive weight schemes or reference-point methods (e.g., MOEA/D, R-NSGA-II) could yield more uniform Pareto-front coverage across diverse operator preferences. Third, robot battery capacity, charging station locations, and task priority levels represent realistic constraints absent from the current model; incorporating them would require feasibility-repair operators and constraint-dominance ranking in the NSGA-II selection step. Fourth, elevator contention—where multiple robots compete for a single elevator—introduces queuing delays not captured by the fixed elevator-cost parameter; modelling this as a resource-constrained scheduling problem would better reflect physical deployment. Finally, validating the framework on physical robot platforms or high-fidelity simulation environments (e.g., ROS 2 + Gazebo) would bridge the gap between theoretical optimisation and practical hotel-service robotics.

ACKNOWLEDGMENT

The authors thank the reviewers for their constructive feedback. This work was carried out as part of an undergraduate project in the Departments of Media Engineering and Technology and Mechatronics and Robotics Engineering, German University in Cairo (GUC).

REFERENCES

- [1] E. Çeltek, "Unmanned smart hotel: applications and examples," *World-wide Hospitality and Tourism Themes*, vol. 16, no. 5, pp. 611–626, 2024.
- [2] A. Ghoneim, A. Elsoda, H. Abdelmawla, K. Elbadawy, L. Tamer, L. Tarek, J. Magdy, and O. Shehata, "Metaheuristic optimization for multi-AGV path planning in search and rescue missions," in *2025 15th International Conference on Electrical Engineering (ICEENG)*. IEEE, 2025, pp. 1–6.
- [3] F. Ahmed, H. Amr, M. Khalil, M. Mohamed, Y. Amir, Z. Yasser, J. Magdy, and O. Shehata, "Route to clean cities: Waste collection optimization with UGVs," in *2025 15th International Conference on Electrical Engineering (ICEENG)*. IEEE, 2025, pp. 1–6.
- [4] S. Kirkpatrick, C. Gelatt, and M. Vecchi, "Optimization by simulated annealing," *Science (New York, N.Y.)*, vol. 220, pp. 671–80, 06 1983.
- [5] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, 1st ed. USA: Addison-Wesley Longman Publishing Co., Inc., 1989.
- [6] X.-F. Liu, B.-C. Lin, Z.-H. Zhan, S.-W. Jeon, and J. Zhang, "An efficient ant colony system for multi-robot task allocation with large-scale cooperative tasks and precedence constraints," in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2021, pp. 1–8.
- [7] Y. Li, W. Huang, R. Wu, and K. Guo, "An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem," *Applied Soft Computing*, vol. 95, p. 106544, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S156849462030483X>
- [8] B. Pradhan, V. Goswami, R. K. Barik, and S. Sahana, "An integrated strategy-based game-theoretic model and decentralized queueing system for mobile multi-robot task coordination," *Decision Analytics Journal*, vol. 7, p. 100254, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772662223000942>
- [9] T. Dong, P. Wang, F. Xue, Y. Geng, and Z. Cui, "Adaptive hybrid response mechanism for dynamic multi-objective optimization and its application in multi-robot task allocation," *Swarm and Evolutionary Computation*, vol. 98, p. 102123, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2210650225002810>
- [10] S. Kim, H. Jang, J. Ha, D. Lee, Y. Ha, and Y. E. Song, "Efficient graph-based multi-story path planning with optimized elevator selection for indoor delivery robots," *Electronics*, vol. 14, no. 5, 2025. [Online]. Available: <https://www.mdpi.com/2079-9292/14/5/982>
- [11] L. Han, J. Ding, S. Liu, and M. Meng, "The path planning problem of robotic delivery in multi-floor hotel environments," *Sensors*, vol. 25, no. 6, 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/25/6/1783>
- [12] X. Bai, C. Li, C. Li, A. Khan, T. Zhang, and B. Zhang, "Multi-robot task assignment for serving people quarantined in multiple hotels during COVID-19 pandemic," *Quantitative Imaging in Medicine and Surgery*, vol. 13, no. 3, pp. 1802–1813, 2023.
- [13] M. Elsheikh, A. Yonis, J. Magdy, and O. Shehata, "Advancing sustainable energy: Analyzing stochastic algorithms and Q-learning for wind farm layout optimization," in *2024 5th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*, 2024, pp. 128–132.